

M V C

Model–View–Controller Pattern

Model, View, Controller

DEFINITION

Separate data (Model), presentation (View) and input handling (Controller) so they evolve independently.

MVC splits a screen into three roles joined by a one-way cycle: the Controller turns user input into Model changes, the Model holds state and business rules, the View renders that state back to the user. The point isn't the three boxes — it's that each role changes for a different reason, so you can edit one without disturbing the others.

WHY IT MATTERS

When input handling, state, and rendering live in one blob, every visual tweak risks the business rules and every rule change risks the screen — and front-end and back-end work can't proceed in parallel. MVC draws seams so each concern evolves on its own schedule and two teams stop colliding in the same file.

— How it works

Model

State plus the business rules that govern it. Knows nothing about the UI — it's the part you'd keep if the screen were replaced by a JSON API.

View

Renders the model's state for the user. Ideally passive: it displays and forwards events, it doesn't decide.

Controller

Turns user input into model operations and picks the next view. The translator between UI events and the domain — keep it thin.

HOW TO APPLY

1. Put state and business rules in the Model — no HTTP, no HTML there.
2. Keep the View rendering-only; hand it a ready view-model, don't let it query the domain.
3. In the Controller: validate input, call one domain operation, choose the response — nothing more.
4. When a controller starts growing logic, push it down into a Model service.

LITMUS TEST

Point at a line: is it deciding what the data is (Model), how it looks (View), or what to do with input (Controller)? If one method does two of those, split it.

— Smell → fix

A route handler that validates, persists, and renders — all three roles fused:

```
app.post('/orders', (req, res) => {  
  const total = req.body.qty * req.body.price // rules  
  db.orders.insert({ total }) // persistence  
  res.send(renderPage(total)) // view  
})  
// one handler owns rules + persistence + rendering
```

↓ SPLIT THE THREE ROLES

```
@Controller('orders')  
class OrdersController {  
  post(@Body() dto: NewOrder) {  
    return this.orders.place(dto) // Service = Model rules  
  }  
} // controller: input only; the view renders the result
```

The controller now only translates HTTP into a domain call and hands state back. Rules live in the service (Model); rendering is the View's job. When a controller starts accruing logic, push it down — fat controllers are the classic MVC smell.

USE WHEN

- ✓ Server-rendered web apps
- ✓ Classic UI frameworks

AVOID WHEN

- ▲ Tiny apps
- ▲ Fat controllers used as a dumping ground

— Trade-offs & pitfalls

PROS

- + Separation of concerns
- + Parallel work across layers

CONS

- − Controller bloat
- − View–Model coupling creep

COMMON PITFALLS

- ▲ Fat controllers: business logic creeps in until the controller becomes the app.
- ▲ Smart views: templates that query the database or branch on domain rules.
- ▲ Treating MVC as three folders rather than three reasons to change.

WHERE YOU SEE IT

- ▶ Rails, Spring MVC, ASP.NET MVC — request to controller to model to view.
- ▶ NestJS controllers delegating to services (Model) and returning DTOs.
- ▶ Server-rendered pages where the controller assembles a view-model for the template.

SEE ALSO

Controller	GRASP names the same role — the first object that handles a system event
SoC	MVC is Separation of Concerns applied to the UI layer
MVP	tighten the loop: the presenter drives a fully passive view
MVVM	replace manual view updates with data binding to a view-model

— Interview & sources

Where do business rules belong in MVC?

In the Model, never the Controller. The controller only routes input to the model and selects a view; logic creeping into controllers is the textbook MVC anti-pattern.

Can the View read the Model directly?

In classic (Smalltalk) MVC, yes — the view observes the model and re-renders. Server-side web MVC usually hands a view-model from the controller to the template instead.

Why do controllers get fat, and how do you keep them thin?

Because every feature seems to "just need one more if". Keep them thin by treating the controller as a translator: validate, call a single domain service, select a response. Logic that branches on business state belongs in the model.

MVC vs MVP vs MVVM?

Same goal, different view binding: MVC's controller mediates input, MVP's presenter drives a passive view 1:1, MVVM binds the view to a view-model (often two-way).

REMEMBER

Input goes to the Controller, state lives in the Model, the View only renders.

SOURCES

Trygve Reenskaug — original MVC notes, Xerox PARC (1978-79)

Martin Fowler — "GUI Architectures" (2006): MVC and its descendants

Rails, Spring MVC, NestJS — server-side MVC in practice